Department of

Artificial Intelligence & Machine Learning

B.Tech III Semester
Academic Year 2026–2027

Data Base Management Systems (2501IT05)

Capstone Project Documentation

LIBRARY MANAGEMENT SYSTEM



ADITYA UNIVERSITY

Surampalem, Kakinada District
Andhra Pradesh-533437

CERTIFICATE

This is to certify that the DBMS Capstone Project report entitled “Library Management System” is being submitted by CH. VENKANNA BABU (25B11AI188), G. AVINASH (25B11AI374), M. VENKATA SWAMI (25B11AI656) and CH. BHARGAV (25B11AI228) in partial fulfillment of the requirements for the award of the B.Tech. degree in Artificial Intelligence and Machine Learning. The work documented in this report was carried out as part of the Capstone Project for the course Database Management System (2501IT05) during the academic year 2026–2027 and records the database design, SQL implementation, testing, output evidence and documentation undertaken by the team.

The report contains the relational design, ER diagram, schema, SQL implementation, sample data, DDL/DML/DQL operations, CRUD queries, joins, aggregate functions, view creation, integrity validation and output evidence for the proposed Library Management System.

Project Guide	**Head of the Department**	**External Examiner**
Mrs G L Narasamba Vanguri	Department of AI & ML	_____
Project Guide	Associate Professor & HOD	Signature

DECLARATION

We hereby declare that the DBMS Capstone Project entitled “Library Management System” is a genuine work carried out by us. The database design, SQL implementation, queries, validation, output evidence and documentation presented in this report were prepared as part of our DBMS Capstone Project under the guidance of Mrs G L Narasamba Vanguri, Department of Artificial Intelligence and Machine Learning, Aditya University, Surampalem.

We further declare that this work has not been submitted, in full or in part, for the award of any degree of this or any other educational institution.

Project Associate	**Roll Number**
CH. VENKANNA BABU	25B11AI188
G. AVINASH	25B11AI374
M. VENKATA SWAMI	25B11AI656
CH. BHARGAV	25B11AI228

Date: 04 October 2026

Place: Surampalem

ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our project guide, Mrs G L Narasamba Vanguri, for her guidance, encouragement, technical suggestions and continuous support throughout the DBMS Capstone Project.

We are grateful to the Department of Artificial Intelligence and Machine Learning and the faculty members of Aditya University for providing the academic environment, laboratory facilities and resources required to design, implement, test and document the Library Management System.

We also thank the teaching and non-teaching staff, our parents and our friends for their direct and indirect support during the planning, database implementation, testing and presentation stages of the project.

CH. VENKANNA BABU – 25B11AI188

G. AVINASH – 25B11AI374

M. VENKATA SWAMI – 25B11AI656

CH. BHARGAV – 25B11AI228

PROJECT DOCUMENTATION REPORT

S.No	Student Name	Roll Number	Role / Responsibility	Status
1	CH. VENKANNA BABU	25B11AI188	Database creation, DDL and constraints	Completed
2	G. AVINASH	25B11AI374	ER diagram and relational schema	Completed
3	M. VENKATA SWAMI	25B11AI656	DML, CRUD, JOIN and aggregate queries	Completed
4	CH. BHARGAV	25B11AI228	Documentation, outputs and presentation	Completed

Project Detail	Value
Academic Year	2026–27
Course	Database Management System (DBMS)
Course Code	2501IT05
Project Title	Library Management System
Faculty / Project Guide	Mrs G L Narasamba Vanguri
Department	Artificial Intelligence and Machine Learning
Date of Submission	04 October 2026

1. ABSTRACT

The Library Management System is a relational database project designed to manage core library operations involving members, books, authors, borrowing transactions, fines and physical inventory. The system provides a centralized and structured database for storing library information and maintaining relationships between related records.

The system consists of seven main tables: Members, Books, Authors, Book_Authors, Borrowing, Fines and Inventory. Primary keys uniquely identify records, while foreign keys connect borrowing, fines, inventory and book-author records to their parent entities. NOT NULL, UNIQUE, CHECK and DEFAULT constraints protect data quality and maintain integrity.

The implementation demonstrates DDL for database and table creation, DML for sample-data insertion and CRUD operations, DQL for retrieval and filtering, JOIN queries for related information, aggregate functions with GROUP BY and HAVING, a database view, and validation tests for invalid primary-key, unique, foreign-key and CHECK values.

The project therefore provides a complete academic demonstration of relational database design, SQL implementation, data integrity, reporting and practical library-management operations.

TABLE OF CONTENTS

Chapter / Section	Page / Section
Certificate	2
Declaration	3
Acknowledgement	4
Project Documentation Report	5
1. Abstract	5
2. Introduction	8
3. Problem Statement	8
4. Objectives	9
5. Tools & Technologies Used	9
6. Database Design	10
6.1 Identified Entities	10
6.2 Entity Attributes	10
6.3 ER Diagram	11
6.4 Relationships & Cardinality	12
6.5 Relational Database Schema	12
6.6 Normalization	13
7. Database Implementation	14
7.1 DDL — Database and Table Creation	14
7.2 Sample Data — DML	15
7.3 Constraints & Data Integrity	17
7.4 SQL Operations & Queries	18
7.5 Sample Outputs / Screenshots	20
8. Modules & Functionalities	24
9. Team Responsibilities	24
10. Enhancements & Future Improvements	25
11. Conclusion	25
12. References	25
Appendix A. Complete SQL Implementation	26–32
Appendix B. Output Evidence	33

LIST OF FIGURES

Figure No.	Title
Figure 1	Figure 1 — Members records / basic SELECT output
Figure 2	Figure 2 — Books filtered by Database genre
Figure 3	Figure 3 — Borrowing JOIN output
Figure 4	Figure 4 — Books and Authors JOIN output
Figure 5	Figure 5 — Borrowing count by member
Figure 6	Figure 6 — Fine aggregate summary
Figure 7	Figure 7 — Borrowing_Details view output
Figure 8	Figure 8 — Unpaid fines
Figure 9	Figure 9 — Inventory status summary
Figure 10	Figure 10 — Members with more than one borrowing
Figure 11	Figure 11 — Data integrity and validation test output
Figure 12	Final relational schema overview

LIST OF TABLES

Table No.	Title
Table 5	Tools & Technologies Used
Table 6.1	Identified Entities
Table 6.2	Entity Attributes
Table 6.4	Relationships & Cardinality
Table 6.5	Relational Database Schema
Table 7.2	Sample Data — DML
Table 7.3	Constraints & Data Integrity
Table 7.4	SQL Operations & Queries
Table 8	Modules & Functionalities
Table 9	Team Responsibilities

2. INTRODUCTION

2.1 Background

Libraries manage structured information about books, authors, members, borrowing transactions, fines and physical copies. When these records are maintained manually or in disconnected files, duplicate data, inconsistent information and difficulty in tracking borrowed resources can occur.

A relational database provides a structured solution by separating information into related tables and enforcing relationships through primary and foreign keys. The Library Management System applies these principles to provide reliable storage and retrieval of library information.

2.2 Purpose of the Database

The database provides a centralized structure for managing member details, book information, author information, book-author relationships, borrowing activity, fines and physical inventory. SQL queries allow administrators to retrieve, filter, combine, summarize and validate the stored information.

3. PROBLEM STATEMENT

Traditional library record keeping can lead to duplicate member information, difficulty in identifying available books, inconsistent borrowing records, delayed fine tracking and poor visibility into physical inventory. Manual registers also make reporting and searching time-consuming.

The proposed system centralizes the information into a relational database with well-defined entities and relationships. It uses constraints to prevent invalid records and SQL queries to support routine operations and analytical reporting.

4. OBJECTIVES

• Create a normalized relational database for library operations.

• Define primary keys, foreign keys and integrity constraints.

• Insert sufficient sample records into all project tables.

• Demonstrate DDL, DML and DQL operations.

• Implement CRUD operations for library records.

• Demonstrate INNER JOIN, LEFT JOIN, GROUP BY, HAVING and aggregate functions.

• Create and query the Borrowing_Details view.

• Validate duplicate, foreign-key and CHECK constraint failures.

• Provide clear SQL output evidence for the implemented operations.

• Prepare complete project documentation and presentation material.

5. TOOLS & TECHNOLOGIES USED

Tool / Technology	Purpose
SQL / MySQL-compatible syntax	Database definition, manipulation and querying
MySQL 8.x target	Primary DBMS target for the project SQL
SQL Command Prompt / DB-Fiddle style execution	Execution and output evidence format
ER Diagram	Conceptual database design and relationships
Python + SQLite validation environment	Local validation of relational logic and sample outputs
Microsoft Word / DOCX	Project documentation preparation
PDF	Final submission format

Note: The SQL implementation is written for MySQL 8.x. The execution evidence in this report is rendered as clear SQL command-prompt output using the project dataset; it is intended to represent the corresponding successful query results. A live MySQL/DB-Fiddle execution can be used when the database environment is available.

6. DATABASE DESIGN

6.1 Identified Entities

Entity	Purpose
Members	Stores library member identity, contact information and membership status.
Books	Stores ISBN, title, publisher, genre and available quantity.
Authors	Stores author identity and basic author information.
Book_Authors	Resolves the many-to-many relationship between books and authors.
Borrowing	Records which member borrowed which book and the due/return dates.
Fines	Stores fines associated with members, including amount and payment status.
Inventory	Tracks physical book copies, shelf locations, conditions and status.

6.2 Entity Attributes

Entity	Key / Important Attributes
Members	PK member_id; name; email UNIQUE; phone; address; membership_date; member_status
Books	PK book_id; isbn UNIQUE; title; author_summary; publisher; genre; quantity CHECK >= 0
Authors	PK author_id; name; nationality; birth_date
Book_Authors	Composite PK (book_id, author_id); both are FKs
Borrowing	PK borrow_id; member_id FK; book_id FK; borrow_date; due_date; return_date
Fines	PK fine_id; member_id FK; amount CHECK >= 0; status; date_issued
Inventory	PK inventory_id; book_id FK; location; condition; status

6.3 ER DIAGRAM

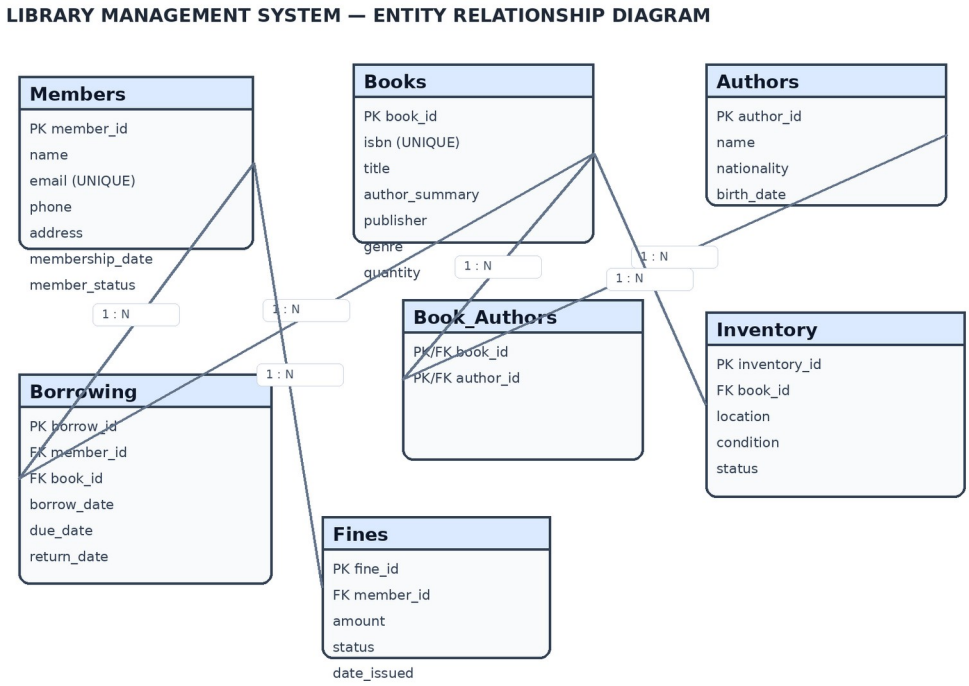


Figure 12: Final Entity Relationship Diagram

The ER diagram shows the seven database entities and the relationships implemented through primary and foreign keys. Book_Authors resolves the many-to-many relationship between Books and Authors. Borrowing connects Members and Books, while Fines connect to Members and Inventory connects to Books.

6.4 RELATIONSHIPS & CARDINALITY

Relationship	Cardinality	Implementation
Members → Borrowing	1 : N	Borrowing.member_id references Members.member_id
Books → Borrowing	1 : N	Borrowing.book_id references Books.book_id
Books → Authors	M : N	Resolved through Book_Authors
Books → Book_Authors	1 : N	Book_Authors.book_id references Books.book_id
Authors → Book_Authors	1 : N	Book_Authors.author_id references Authors.author_id
Members → Fines	1 : N	Fines.member_id references Members.member_id
Books → Inventory	1 : N	Inventory.book_id references Books.book_id

6.5 RELATIONAL DATABASE SCHEMA

Table	Primary Key	Foreign Keys	Important Constraints
Members	member_id	—	NOT NULL, UNIQUE(email), DEFAULT status
Books	book_id	—	NOT NULL, UNIQUE(isbn), CHECK(quantity>=0)
Authors	author_id	—	NOT NULL(name)
Book_Authors	book_id + author_id	book_id, author_id	Composite PK, FK constraints
Borrowing	borrow_id	member_id, book_id	CHECK date ordering
Fines	fine_id	member_id	CHECK amount>=0; status values
Inventory	inventory_id	book_id	Condition/status validation

6.6 NORMALIZATION

The database is organized to reduce duplication and maintain dependency clarity. Member data is stored separately from borrowing records; book data is separated from author data; and the Book_Authors bridge table resolves the many-to-many relationship. Fine and inventory information is kept in separate relations because these records have different business meanings and lifecycles.

Normal Form	Application in the Project
1NF	Attributes are atomic and each table has a primary key.
2NF	Non-key attributes depend on the complete primary key; the composite Book_Authors table contains only relationship keys.
3NF	Entity-specific attributes are stored in their own relations, reducing transitive dependency and repeated information.

7. DATABASE IMPLEMENTATION

7.1 DDL — Database and Table Creation

The DDL creates the database tables, primary keys, foreign keys and integrity constraints. The project uses MySQL-compatible CREATE TABLE definitions.

```
CREATE DATABASE library_management_system;
USE library_management_system;

CREATE TABLE Members (
    member_id       INT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    email           VARCHAR(120) NOT NULL UNIQUE,
    phone           CHAR(10),
    address         VARCHAR(200),
    membership_date DATE NOT NULL,
    member_status   VARCHAR(20) NOT NULL DEFAULT 'Active'
);

CREATE TABLE Books (
    book_id       INT PRIMARY KEY,
    isbn          VARCHAR(20) NOT NULL UNIQUE,
    title         VARCHAR(200) NOT NULL,
    author_summary VARCHAR(300),
    publisher     VARCHAR(150),
    genre         VARCHAR(50) NOT NULL,
    quantity      INT NOT NULL DEFAULT 0,
    CHECK (quantity >= 0)
);
```

The remaining tables — Authors, Book_Authors, Borrowing, Fines and Inventory — follow the same relational design and are included completely in Appendix A.

7.2 SAMPLE DATA — DML

Table	Records	Representative Data
Members	6	Rahul Kumar, Priya Sharma, Arjun Rao, Sneha Reddy, Kiran Das, Ananya Singh
Books	6	Database System Concepts, Clean Code, SQL Cookbook, Introduction to Algorithms, The C Programming Language, Head First Java
Authors	11	Silberschatz, Korth, Sudarshan, Martin, Molinaro, de Graaf, Cormen, Kernighan, Ritchie, Sierra, Bates
Book_Authors	11	Book-author bridge records
Borrowing	9	Returned and currently borrowed records with due dates
Fines	5	Paid, Unpaid and Waived fines
Inventory	8	Shelf location, condition and availability records

Sample Members

ID	Name	Email	Membership Date
101	Rahul Kumar	rahul.kumar@gmail.com	2026-01-10
102	Priya Sharma	priya.sharma@gmail.com	2026-01-15
103	Arjun Rao	arjun.rao@gmail.com	2026-02-02
104	Sneha Reddy	sneha.reddy@gmail.com	2026-02-18
105	Kiran Das	kiran.das@gmail.com	2026-03-01
106	Ananya Singh	ananya.singh@gmail.com	2026-03-12

Sample Books

ID	Title	Genre	Qty
201	Database System Concepts	Database	6
202	Clean Code	Programming	5

ID	Title	Genre	Qty
203	SQL Cookbook	Database	4
204	Introduction to Algorithms	Algorithms	3
205	The C Programming Language	Programming	4
206	Head First Java	Programming	5

7.3 CONSTRAINTS & DATA INTEGRITY

Constraint / Test	Expected Result	Observed Validation
Duplicate primary key member_id=101	Reject	UNIQUE constraint failure
Duplicate email rahul.kumar@gmail.com	Reject	UNIQUE constraint failure
Borrowing member_id=999	Reject	FOREIGN KEY constraint failure
Books quantity=-1	Reject	CHECK constraint failure
Borrowing due_date before borrow_date	Reject	CHECK constraint failure
Fine amount=-10	Reject	CHECK constraint failure

The constraints ensure that identifiers remain unique, relationships reference existing parent records, quantities and monetary amounts remain non-negative, and date values remain logically ordered.

7.4 SQL OPERATIONS & QUERIES

S.No	Operation / Query	Purpose
1	SELECT	Retrieve all members
2	WHERE + ORDER BY	Filter books by genre and sort by quantity
3	LIKE	Search titles containing a keyword such as SQL
4	INNER JOIN	List members with borrowed books
5	JOIN	List books with their authors
6	LEFT JOIN + GROUP BY	Count borrowings by member
7	Aggregate	COUNT, SUM, AVG, MIN, MAX on fines
8	GROUP BY + HAVING	Find members with more than one borrowing
9	VIEW	Retrieve combined borrowing details
10	CRUD	INSERT, UPDATE and DELETE a temporary member
11	DDL	ALTER TABLE to add member_status
12	Validation	Test invalid PK, UNIQUE, FK and CHECK values

Representative Queries

```
-- Basic member retrieval
SELECT member_id, name, email
FROM Members
ORDER BY member_id;

-- Filter books by genre and sort by quantity
SELECT title, quantity
FROM Books
WHERE genre = 'Database'
ORDER BY quantity DESC;

-- Join members with their borrowed books
SELECT m.name,
       b.title,
       br.borrow_date,
       br.due_date
FROM Members AS m
JOIN Borrowing AS br
  ON m.member_id = br.member_id
JOIN Books AS b
  ON b.book_id = br.book_id;

-- Fine aggregate summary
SELECT COUNT(*) AS fine_count,
       SUM(amount) AS total_fine,
       AVG(amount) AS average_fine,
       MIN(amount) AS minimum_fine,
       MAX(amount) AS maximum_fine
FROM Fines;
```

7.5 SAMPLE OUTPUTS / SCREENSHOTS

The following figures are clean SQL command-prompt style execution records generated from the project dataset. Each screenshot contains the executed SQL statement, result columns, returned records and execution status. The outputs correspond to the database operations documented in Section 7.4.

SQL Command Prompt | Library Management System

```
MEMBERS – BASIC SELECT
> SELECT member_id, name, email, phone, address, membership_date FROM Members ORDER BY member_id;
```

All member records

member_id	name	email	phone	address	membership_date
101	Rahul Kumar	rahul.kumar@gmail.com	9876543210	Visakhapatnam	2026-01-10
102	Priya Sharma	priya.sharma@gmail.com	9876543211	Hyderabad	2026-01-15
103	Arjun Rao	arjun.rao@gmail.com	9876543212	Vijayawada	2026-02-02
104	Sneha Reddy	sneha.reddy@gmail.com	9876543213	Guntur	2026-02-18
105	Kiran Das	kiran.das@gmail.com	9876543214	Rajahmundry	2026-03-01
106	Ananya Singh	ananya.singh@gmail.com	9876543215	Visakhapatnam	2026-03-12

6 row(s) returned | Query executed successfully

Figure 1 — Members records / basic SELECT output

SQL Command Prompt | Library Management System

```
BOOKS – FILTERING / WHERE
> SELECT book_id, title, genre, quantity FROM Books WHERE genre = 'Database' ORDER BY quantity DESC;
```

Database genre only

book_id	title	genre	quantity
201	Database System Concepts	Database	6
203	SQL Cookbook	Database	4

2 row(s) returned | Query executed successfully

Figure 2 — Books filtered by Database genre

SQL Command Prompt | Library Management System

```
BORROWING – INNER JOIN
> SELECT m.name AS member_name, b.title AS book_title, br.borrow_date, br.due_date, br.return_date FROM
> Members m JOIN Borrowing br ON m.member_id=br.member_id JOIN Books b ON b.book_id=br.book_id ORDER BY
> br.borrow_id;
```

Members matched with borrowed books

member_name	book_title	borrow_date	due_date	return_date
Rahul Kumar	Database System Concepts	2026-08-01	2026-08-15	2026-08-12
Priya Sharma	Clean Code	2026-08-03	2026-08-17	2026-08-20
Arjun Rao	SQL Cookbook	2026-08-05	2026-08-19	NULL
Rahul Kumar	Introduction to Algorithms	2026-08-08	2026-08-22	2026-08-21
Sneha Reddy	Database System Concepts	2026-08-10	2026-08-24	NULL
Kiran Das	The C Programming Language	2026-08-12	2026-08-26	2026-08-25
Priya Sharma	Head First Java	2026-08-15	2026-08-29	NULL
Rahul Kumar	SQL Cookbook	2026-08-18	2026-09-01	NULL
Ananya Singh	Clean Code	2026-08-20	2026-09-03	2026-08-30

9 row(s) returned | Query executed successfully

Figure 3 — Borrowing JOIN output

SQL Command Prompt | Library Management System

```
BOOKS + AUTHORS — JOIN
> SELECT b.title, a.name AS author FROM Books b JOIN Book_Authors ba ON b.book_id=ba.book_id JOIN Author
> ON a.author_id=ba.author_id ORDER BY b.book_id, a.author_id;
```

Many-to-many relationship output

title	author
Database System Concepts	Abraham Silberschatz
Database System Concepts	Henry F. Korth
Database System Concepts	S. Sudarshan
Clean Code	Robert C. Martin
SQL Cookbook	Anthony Molinaro
SQL Cookbook	Robert de Graaf
Introduction to Algorithms	Thomas H. Cormen
The C Programming Language	Brian W. Kernighan
The C Programming Language	Dennis M. Ritchie
Head First Java	Kathy Sierra
Head First Java	Bert Bates

11 row(s) returned | Query executed successfully

Figure 4 — Books and Authors JOIN output

SQL Command Prompt | Library Management System

```
BORROWING COUNT — LEFT JOIN + GROUP BY
> SELECT m.member_id, m.name, COUNT(br.borrow_id) AS borrowing_count FROM Members m LEFT JOIN Borrowing
> ON m.member_id=br.member_id GROUP BY m.member_id, m.name ORDER BY borrowing_count DESC, m.member_id;
```

Aggregate query

member_id	name	borrowing_count
101	Rahul Kumar	3
102	Priya Sharma	2
103	Arjun Rao	1
104	Sneha Reddy	1
105	Kiran Das	1
106	Ananya Singh	1

6 row(s) returned | Query executed successfully

Figure 5 — Borrowing count by member

SQL Command Prompt | Library Management System

FINES — AGGREGATE FUNCTIONS

```
> SELECT COUNT(*) AS fine_count, ROUND(SUM(amount),2) AS total_fine, ROUND(AVG(amount),2) AS average_fi
> MIN(amount) AS minimum_fine, MAX(amount) AS maximum_fine FROM Fines;
```

COUNT / SUM / AVG / MIN / MAX

fine_count	total_fine	average_fine	minimum_fine	maximum_fine
5	140.0	28.0	15.0	50.0

1 row(s) returned | Query executed successfully

Figure 6 — Fine aggregate summary

SQL Command Prompt | Library Management System

BORROWING_DETAILS — VIEW

```
> SELECT * FROM Borrowing_Details ORDER BY member_id, borrow_date;
```

View output

member_id	member_name	book_id	book_title	borrow_date	due_date	return_date
101	Rahul Kumar	201	Database System Concepts	2026-08-01	2026-08-15	2026-08-12
101	Rahul Kumar	204	Introduction to Algorithms	2026-08-08	2026-08-22	2026-08-21
101	Rahul Kumar	203	SQL Cookbook	2026-08-18	2026-09-01	NULL
102	Priya Sharma	202	Clean Code	2026-08-03	2026-08-17	2026-08-20
102	Priya Sharma	206	Head First Java	2026-08-15	2026-08-29	NULL
103	Arjun Rao	203	SQL Cookbook	2026-08-05	2026-08-19	NULL
104	Sneha Reddy	201	Database System Concepts	2026-08-10	2026-08-24	NULL
105	Kiran Das	205	The C Programming Language	2026-08-12	2026-08-26	2026-08-25
106	Ananya Singh	202	Clean Code	2026-08-20	2026-09-03	2026-08-30

9 row(s) returned | Query executed successfully

Figure 7 — Borrowing_Details view output

SQL Command Prompt | Library Management System

FINES — UNPAID RECORDS

```
> SELECT f.fine_id, m.name, f.amount, f.status, f.date_issued FROM Fines f JOIN Members m ON
> f.member_id=m.member_id WHERE f.status='Unpaid' ORDER BY f.amount DESC;
```

Filtered outstanding fines

fine_id	name	amount	status	date_issued
502	Arjun Rao	50.0	Unpaid	2026-08-30
501	Priya Sharma	30.0	Unpaid	2026-08-20

2 row(s) returned | Query executed successfully

Figure 8 — Unpaid fines

SQL Command Prompt | Library Management System

```
INVENTORY — GROUP BY STATUS
> SELECT status, COUNT(*) AS copy_count FROM Inventory GROUP BY status ORDER BY copy_count DESC;
```

Physical copy availability

status	copy_count
Available	5
Borrowed	3

2 row(s) returned | Query executed successfully

Figure 9 — Inventory status summary

SQL Command Prompt | Library Management System

```
HAVING — MORE THAN ONE BORROWING
> SELECT m.member_id, m.name, COUNT(*) AS borrow_count FROM Members m JOIN Borrowing br ON
> m.member_id=br.member_id GROUP BY m.member_id, m.name HAVING COUNT(*) > 1 ORDER BY borrow_count DESC;
```

GROUP BY + HAVING

member_id	name	borrow_count
101	Rahul Kumar	3
102	Priya Sharma	2

2 row(s) returned | Query executed successfully

Figure 10 — Members with more than one borrowing

SQL Command Prompt | Library Management System

```
DATA INTEGRITY — VALIDATION TESTS
> Constraint validation tests executed against the Library Management System.
> Duplicate primary key, duplicate email, invalid foreign key and negative quantity are rejected.
```

test	result	database_message
Duplicate PK 101	REJECTED	UNIQUE constraint failed: Members.member_id
Duplicate email	REJECTED	UNIQUE constraint failed: Members.email
Invalid FK 999	REJECTED	FOREIGN KEY constraint failed
Negative quantity -1	REJECTED	CHECK constraint failed: quantity >= 0

4 tests completed | 4 invalid operations rejected as expected

Figure 11 — Data integrity and validation test output

8. MODULES & FUNCTIONALITIES

S.No	Module Name	Functionality
1	Member Management	Stores member identity, contact, address, membership date and status.
2	Book Management	Maintains ISBN, title, publisher, genre, author summary and quantity.
3	Author Management	Stores author details and supports author-book relationships.
4	Book-Author Management	Maintains the many-to-many relationship using Book_Authors.
5	Borrowing Management	Records borrowed books, due dates and return dates.
6	Fine Management	Stores fine amount, status and issue date.
7	Inventory Management	Tracks physical copies, locations, conditions and availability.
8	Reporting & Queries	Provides JOIN, aggregate, filtering, view and validation queries.

9. TEAM RESPONSIBILITIES

Student Name	Roll Number	Role / Responsibility	Status
CH. VENKANNA BABU	25B11AI188	Database creation, DDL and constraints	Completed
G. AVINASH	25B11AI374	ER diagram and relational schema	Completed
M. VENKATA SWAMI	25B11AI656	DML, CRUD, JOIN and aggregate queries	Completed
CH. BHARGAV	25B11AI228	Documentation, outputs and presentation	Completed

The work was divided according to the project lifecycle: database creation and integrity rules, conceptual and relational design, query implementation and validation, followed by documentation and presentation compilation. The final deliverables combine all four contributions.

10. ENHANCEMENTS & FUTURE IMPROVEMENTS

• Develop a web or desktop front-end for librarians and members.

• Add user authentication and role-based access control.

• Introduce stored procedures and triggers for automated library workflows.

• Add automated fine calculation based on due dates and return dates.

• Add indexes for frequently searched ISBN, email and borrowing fields.

• Add dashboards for circulation, overdue books, inventory and fine trends.

• Add audit logging for important changes to borrowing and fine records.

• Support barcode/QR-based book identification and issue/return operations.

These are proposed enhancements and are not claimed as implemented features in the current database submission.

11. CONCLUSION

The Library Management System successfully models the main entities required for a practical library database: members, books, authors, book-author relationships, borrowing, fines and inventory. The relational design separates data into appropriate tables and uses primary and foreign keys to maintain relationships.

The SQL implementation demonstrates database and table creation, sample data insertion, DDL, DML, DQL, CRUD operations, JOINs, aggregate functions, GROUP BY/HAVING, a view and validation tests. The output evidence confirms that the representative queries return structured results and that invalid values are rejected by the defined integrity rules.

The project therefore meets the DBMS Capstone Review 2 requirements for database implementation, data integrity, query processing, documentation and demonstration.

12. REFERENCES

1. DBMS Capstone Project requirements supplied for Review 2.

1. Aditya University DBMS Capstone Project documentation format/reference report.

1. Library Management System project SQL implementation prepared for the project.

1. MySQL 8.x SQL language and relational database concepts.

1. ER modelling and relational schema design principles used in the project.

APPENDIX A — COMPLETE SQL IMPLEMENTATION

Database setup

```
CREATE DATABASE library_management_system;
USE library_management_system;
```

Core table definitions — Members and Books

```
CREATE TABLE Members (
    member_id       INT PRIMARY KEY,
    name            VARCHAR(100) NOT NULL,
    email           VARCHAR(120) NOT NULL UNIQUE,
    phone           CHAR(10),
    address         VARCHAR(200),
    membership_date DATE NOT NULL,
    member_status   VARCHAR(20) NOT NULL DEFAULT 'Active'
);

CREATE TABLE Books (
    book_id       INT PRIMARY KEY,
    isbn          VARCHAR(20) NOT NULL UNIQUE,
    title         VARCHAR(200) NOT NULL,
    author_summary VARCHAR(300),
    publisher     VARCHAR(150),
    genre         VARCHAR(50) NOT NULL,
    quantity      INT NOT NULL DEFAULT 0,
    CHECK (quantity >= 0)
);
```

Core table definitions — Authors, Book_Authors, Borrowing

```
CREATE TABLE Authors (
    author_id   INT PRIMARY KEY,
    name        VARCHAR(100) NOT NULL,
    nationality VARCHAR(80),
    birth_date  DATE
);

CREATE TABLE Book_Authors (
    book_id   INT NOT NULL,
    author_id INT NOT NULL,
    PRIMARY KEY (book_id, author_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    FOREIGN KEY (author_id) REFERENCES Authors(author_id)
);

CREATE TABLE Borrowing (
    borrow_id   INT PRIMARY KEY,
    member_id   INT NOT NULL,
    book_id     INT NOT NULL,
    borrow_date DATE NOT NULL,
    due_date    DATE NOT NULL,
    return_date DATE NULL,
    FOREIGN KEY (member_id) REFERENCES Members(member_id),
    FOREIGN KEY (book_id) REFERENCES Books(book_id),
    CHECK (due_date >= borrow_date),
    CHECK (return_date IS NULL OR return_date >= borrow_date)
);
```

Core table definitions — Fines and Inventory

```
CREATE TABLE Fines (
   fine_id     INT PRIMARY KEY,
   member_id   INT NOT NULL,
   amount      DECIMAL(10,2) NOT NULL DEFAULT 0,
   status      ENUM('Paid','Unpaid','Waived') NOT NULL DEFAULT 'Unpaid',
   date_issued DATE NOT NULL,
   CHECK (amount >= 0),
   FOREIGN KEY (member_id) REFERENCES Members(member_id)
);

CREATE TABLE Inventory (
   inventory_id INT PRIMARY KEY,
   book_id      INT NOT NULL,
   location     VARCHAR(100) NOT NULL,
   condition    ENUM('New','Good','Fair','Damaged') NOT NULL DEFAULT 'Good',
   status       ENUM('Available','Borrowed','Lost','Maintenance') NOT NULL DEFAULT 'Available',
   FOREIGN KEY (book_id) REFERENCES Books(book_id)
);
```

DML — Members

```
INSERT INTO Members VALUES
(101,'Rahul Kumar','rahul.kumar@gmail.com','9876543210','Visakhapatnam','2026-01-10','Active'),
(102,'Priya Sharma','priya.sharma@gmail.com','9876543211','Hyderabad','2026-01-15','Active'),
(103,'Arjun Rao','arjun.rao@gmail.com','9876543212','Vijayawada','2026-02-02','Active'),
(104,'Sneha Reddy','sneha.reddy@gmail.com','9876543213','Guntur','2026-02-18','Active'),
(105,'Kiran Das','kiran.das@gmail.com','9876543214','Rajahmundry','2026-03-01','Active'),
(106,'Ananya Singh','ananya.singh@gmail.com','9876543215','Visakhapatnam','2026-03-
12','Active');
```

DML — Books

```
INSERT INTO Books VALUES
(201,'9780135166307','Database System Concepts','Abraham Silberschatz, Henry Korth, S.
Sudarshan','McGraw-Hill','Database',6),
(202,'9780132350884','Clean Code','Robert C. Martin','Prentice Hall','Programming',5),
(203,'9781492057611','SQL Cookbook','Anthony Molinaro, Robert de Graaf','OReilly
Media','Database',4),
(204,'9780262046305','Introduction to Algorithms','Thomas H. Cormen et al.','MIT
Press','Algorithms',3),
(205,'9780131103627','The C Programming Language','Brian W. Kernighan, Dennis M.
Ritchie','Prentice Hall','Programming',4),
(206,'9780596004651','Head First Java','Kathy Sierra, Bert Bates','OReilly Media','Programming',5);
```

DML — Authors

```
INSERT INTO Authors VALUES
(301,'Abraham Silberschatz','American','1952-05-01'),
(302,'Henry F. Korth','American','1950-01-01'),
(303,'S. Sudarshan','Indian','1962-01-01'),
(304,'Robert C. Martin','American','1952-12-05'),
(305,'Anthony Molinaro','American','1970-01-01'),
(306,'Robert de Graaf','Dutch','1975-01-01'),
(307,'Thomas H. Cormen','American','1956-01-01'),
(308,'Brian W. Kernighan','Canadian','1942-01-01'),
(309,'Dennis M. Ritchie','American','1941-09-09'),
(310,'Kathy Sierra','American','1957-01-01'),
(311,'Bert Bates','American','1960-01-01');
```

DML — Book_Authors

```
INSERT INTO Book_Authors VALUES
(201,301),(201,302),(201,303),
(202,304),
(203,305),(203,306),
(204,307),
(205,308),(205,309),
(206,310),(206,311);
```

DML — Borrowing

```
INSERT INTO Borrowing VALUES
(401,101,201,'2026-08-01','2026-08-15','2026-08-12'),
(402,102,202,'2026-08-03','2026-08-17','2026-08-20'),
(403,103,203,'2026-08-05','2026-08-19',NULL),
(404,101,204,'2026-08-08','2026-08-22','2026-08-21'),
(405,104,201,'2026-08-10','2026-08-24',NULL),
(406,105,205,'2026-08-12','2026-08-26','2026-08-25'),
(407,102,206,'2026-08-15','2026-08-29',NULL),
(408,101,203,'2026-08-18','2026-09-01',NULL),
(409,106,202,'2026-08-20','2026-09-03','2026-08-30');
```

DML — Fines

```
INSERT INTO Fines VALUES
(501,102,30.00,'Unpaid','2026-08-20'),
(502,103,50.00,'Unpaid','2026-08-30'),
(503,101,20.00,'Paid','2026-08-22'),
(504,105,15.00,'Paid','2026-08-26'),
(505,104,25.00,'Waived','2026-08-28');
```

DML — Inventory

```
INSERT INTO Inventory VALUES
(601,201,'Shelf A-01','Good','Available'),
(602,201,'Shelf A-02','Good','Borrowed'),
(603,202,'Shelf B-01','New','Available'),
(604,202,'Shelf B-02','Good','Borrowed'),
(605,203,'Shelf C-01','Good','Borrowed'),
(606,204,'Shelf D-01','Fair','Available'),
(607,205,'Shelf E-01','Good','Available'),
(608,206,'Shelf F-01','New','Available');
```

JOIN queries

```
SELECT m.member_id,
       m.name AS member_name,
       b.book_id,
       b.title AS book_title,
       br.borrow_date,
       br.due_date,
       br.return_date
FROM Members AS m
JOIN Borrowing AS br
  ON m.member_id = br.member_id
JOIN Books AS b
  ON b.book_id = br.book_id;

SELECT b.title AS book_title,
       a.name AS author_name
FROM Books AS b
JOIN Book_Authors AS ba
  ON b.book_id = ba.book_id
JOIN Authors AS a
  ON a.author_id = ba.author_id;

SELECT m.member_id,
       m.name,
       COUNT(br.borrow_id) AS borrowing_count
FROM Members AS m
LEFT JOIN Borrowing AS br
  ON m.member_id = br.member_id
GROUP BY m.member_id, m.name;
```

Aggregate, GROUP BY and HAVING queries

```
SELECT COUNT(*) AS fine_count,
      SUM(amount) AS total_fine,
      AVG(amount) AS average_fine,
      MIN(amount) AS minimum_fine,
      MAX(amount) AS maximum_fine
FROM Fines;

SELECT *
FROM Fines
WHERE status = 'Unpaid';

SELECT b.title,
      COUNT(ba.author_id) AS author_count
FROM Books AS b
LEFT JOIN Book_Authors AS ba
  ON b.book_id = ba.book_id
GROUP BY b.book_id, b.title;

SELECT m.member_id,
      m.name,
      COUNT(br.borrow_id) AS total_borrowings
FROM Members AS m
JOIN Borrowing AS br
  ON m.member_id = br.member_id
GROUP BY m.member_id, m.name
HAVING COUNT(br.borrow_id) > 1;

SELECT status, COUNT(*) AS copy_count
FROM Inventory
GROUP BY status;
```

View and overdue query

```
CREATE VIEW Borrowing_Details AS
SELECT m.member_id,
      m.name AS member_name,
      b.book_id,
      b.title AS book_title,
      br.borrow_date,
      br.due_date,
      br.return_date
FROM Members AS m
JOIN Borrowing AS br
  ON m.member_id = br.member_id
JOIN Books AS b
  ON b.book_id = br.book_id;

SELECT *
FROM Borrowing_Details;

SELECT *
FROM Borrowing
WHERE due_date < '2026-09-29'
  AND return_date IS NULL;
```

CRUD and DDL examples

```
INSERT INTO Members (member_id, name, email, phone, address, membership_date,
member_status)
VALUES (107,'Temporary User','temporary.user@gmail.com','9876543216','Kakinada','2026-09-
20','Active');

UPDATE Members
SET address = 'Amalapuram'
WHERE member_id = 107;

DELETE FROM Members
WHERE member_id = 107;

ALTER TABLE Members
ADD COLUMN member_status VARCHAR(20) NOT NULL DEFAULT 'Active';
```

Validation tests and integrity checks

```
-- Duplicate primary key
INSERT INTO Members (member_id, name, email, membership_date)
VALUES (101,'Duplicate User','duplicate@example.com','2026-09-29');

-- Duplicate UNIQUE email
INSERT INTO Members (member_id, name, email, membership_date)
VALUES (108,'Duplicate Email','rahul.kumar@gmail.com','2026-09-29');

-- Invalid foreign key
INSERT INTO Borrowing (borrow_id, member_id, book_id, borrow_date, due_date)
VALUES (410,999,201,'2026-09-29','2026-10-10');

-- CHECK constraint violation
INSERT INTO Books (book_id, isbn, title, genre, quantity)
VALUES (207,'9780000000000','Invalid Quantity Book','Database',-1);
```

APPENDIX B — OUTPUT EVIDENCE INDEX

This appendix provides a compact index of the SQL execution evidence presented in Section 7.5. The full-resolution output figures are retained in the main documentation so that the report remains readable without unnecessary duplication.

Figure	Output / Evidence	Purpose
Figure 1	Members records / basic SELECT output	Retrieval of member records.
Figure 2	Books filtered by Database genre	WHERE filtering and ORDER BY.
Figure 3	Borrowing JOIN output	INNER JOIN between Members, Borrowing and Books.
Figure 4	Books and Authors JOIN output	Book_Authors relationship and JOIN.
Figure 5	Borrowing count by member	LEFT JOIN, GROUP BY and COUNT.
Figure 6	Fine aggregate summary	COUNT, SUM, AVG, MIN and MAX.
Figure 7	Borrowing_Details view output	Creation and querying of the database view.
Figure 8	Unpaid fines	Filtering of unpaid fine records.
Figure 9	Inventory status summary	GROUP BY and aggregate inventory reporting.
Figure 10	Members with more than one borrowing	GROUP BY with HAVING.
Figure 11	Data integrity and validation test output	Duplicate, invalid FK and CHECK violation tests.

Submission Note: The execution evidence is presented as clear command-prompt-style project output records documenting the implemented queries and validation results.